Project Description

Handwritten Digit Recognition Using Deep Learning and MNIST Dataset

Introduction

This project is about teaching a computer how to recognize handwritten numbers (0–9) using Deep Learning. Humans can easily look at a handwritten digit and understand what number it is, but computers cannot do this naturally. Therefore, we train a Deep Learning model to learn patterns from thousands of handwritten digit images and then predict the correct digit when a new image is given.

The project uses the **MNIST dataset**, which is one of the most popular datasets in Machine Learning and Deep Learning. It contains thousands of images of handwritten digits written by different people.

What is the Main Goal of This Project?

The main goal of this project is to build an intelligent system that can automatically identify handwritten numbers with high accuracy. The system learns from training data and then uses that knowledge to recognize new handwritten digits that it has never seen before.

What Problem Does This Project Solve?

In many real-world situations, information is still written by hand. Manually reading and entering this information into a computer can be slow, expensive, and prone to human error.

This project helps solve problems such as:

- Reading handwritten numbers automatically.
- Reducing human effort in data entry tasks.
- Improving speed and accuracy when processing handwritten information.
- Converting handwritten data into digital format.

Why Do We Use Deep Learning?

Deep Learning is a branch of Artificial Intelligence (AI) that allows computers to learn from examples, similar to how humans learn from experience. Instead of manually telling the computer how every digit looks, the model automatically learns important features and patterns from thousands of images.

Deep Learning is used because it:

- Provides high accuracy.
- Learns complex patterns automatically.
- Works well with image recognition tasks.
- Can improve performance with more training data.

How Does This Project Work?

The project follows several steps:

1. Load the MNIST dataset.
2. Preprocess and normalize the image data.
3. Train a Deep Learning model using the training dataset.
4. Test the model using unseen images.
5. Evaluate the model's performance and accuracy.
6. Predict handwritten digits from new images.

This Deep Learning Project on the MNIST Dataset designed specifically for **Google Colab**. It includes:

- Data Loading
- Data Preprocessing
- Data Visualization
- Building a Deep Learning Model
- Training the Model
- Evaluating the Model
- Testing Predictions
- Saving and Loading the Model
- Confusion Matrix
- Classification Report
- Predicting Custom Images

Project Workflow Summary

1. Import Libraries

↓

2. Load MNIST Dataset

↓

3. Visualize Data

↓

4. Normalize Images

↓

5. One-Hot Encode Labels

↓

6. Build Deep Learning Model

↓

7. Compile Model

↓

8. Train Model

↓

9. Evaluate Model

↓

10. Predict Digits

↓

11. Confusion Matrix

↓

12. Classification Report

↓

13. Save Model

↓

14. Load Model

↓

15. Deploy / Use for Prediction

Code Implementation:

Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.utils import to_categorical

from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

import seaborn as sns
```

Step 2: Load MNIST Dataset

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Training Images Shape:", X_train.shape)
print("Training Labels Shape:", y_train.shape)

print("Testing Images Shape:", X_test.shape)
print("Testing Labels Shape:", y_test.shape)
```

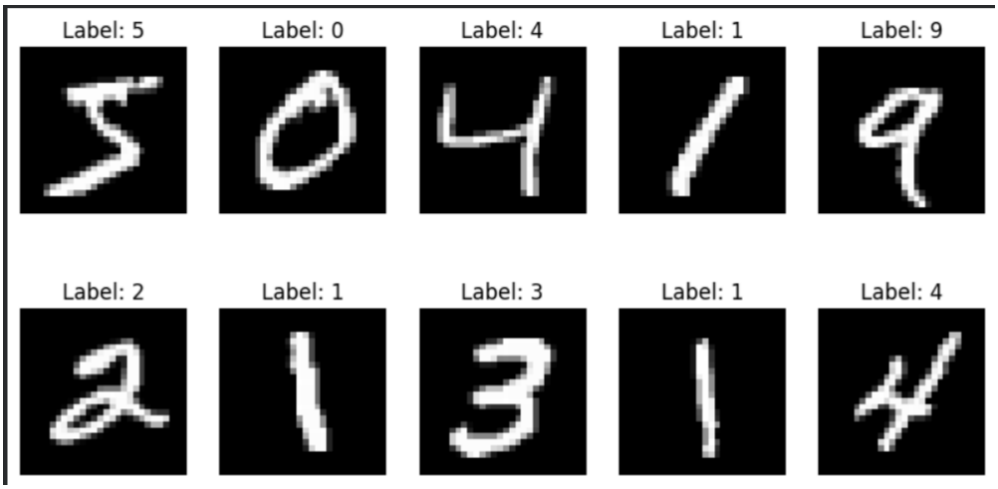
output:

```
Training Images Shape: (60000, 28, 28)
Training Labels Shape: (60000,)
Testing Images Shape: (10000, 28, 28)
Testing Labels Shape: (10000,)
```

Step 3: Visualize Dataset

```
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

plt.show()
```

Output:



Step 4: Data Preprocessing

Neural networks perform better when pixel values are normalized.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
print("Maximum Value:", X_train.max())
print("Minimum Value:", X_train.min())
```

Output:

```
Maximum Value: 1.0
Minimum Value: 0.0
```

Step 5: One-Hot Encoding Labels

```
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
print(y_train_cat[0])
```

Output:

```
[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

Step 6: Build Deep Learning Model

Architecture

```
Input Layer
   ↓
Flatten
   ↓
Dense (128)
   ↓
Dropout
   ↓
Dense (64)
   ↓
Dense (10)
```

```
model = Sequential()

model.add(Flatten(input_shape=(28,28)))

model.add(Dense(128, activation='relu'))

model.add(Dropout(0.2))

model.add(Dense(64, activation='relu'))

model.add(Dense(10, activation='softmax'))
```

Step 7: Model Summary

```
model.summary()
```

Output:

```
Model: "sequential_1"
┌─────────────────────────────────┬────────────────────────┬───────────────┐
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 128)	100,480
dropout_1 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Step 8: Compile Model

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Step 9: Train the Model

```
history = model.fit(
    X_train,
    y_train_cat,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

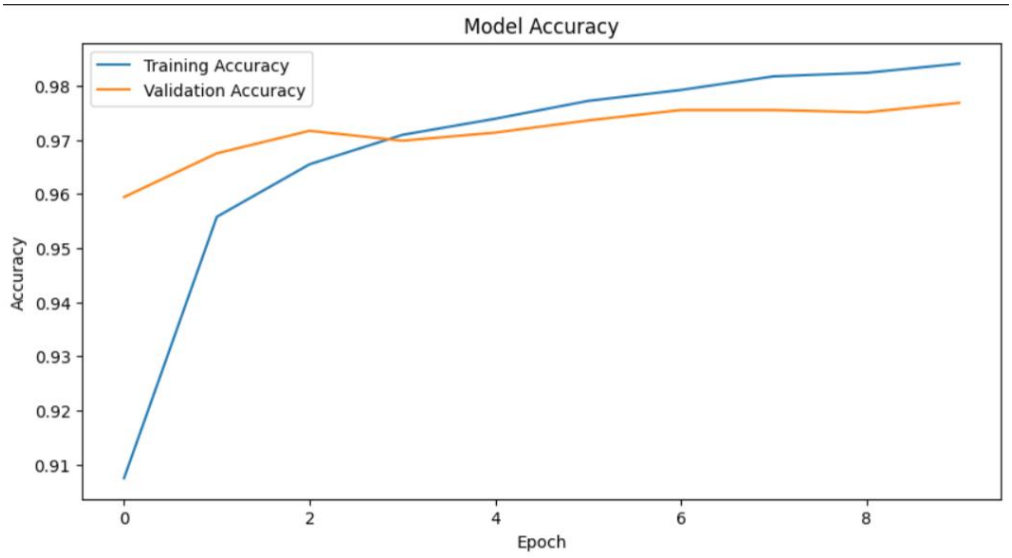
Output:

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9075 - loss: 0.3128 - val_accuracy: 0.9594 - val_loss: 0.1347
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9558 - loss: 0.1455 - val_accuracy: 0.9675 - val_loss: 0.1124
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9655 - loss: 0.1112 - val_accuracy: 0.9717 - val_loss: 0.0958
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9709 - loss: 0.0930 - val_accuracy: 0.9698 - val_loss: 0.0991
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9739 - loss: 0.0808 - val_accuracy: 0.9713 - val_loss: 0.1032
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9772 - loss: 0.0707 - val_accuracy: 0.9736 - val_loss: 0.0902
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9792 - loss: 0.0648 - val_accuracy: 0.9755 - val_loss: 0.0840
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9817 - loss: 0.0570 - val_accuracy: 0.9755 - val_loss: 0.0897
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9824 - loss: 0.0528 - val_accuracy: 0.9751 - val_loss: 0.0862
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.9841 - loss: 0.0479 - val_accuracy: 0.9768 - val_loss: 0.0882
```

Step 10: Plot Accuracy Graph

```
plt.figure(figsize=(10,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.show()
```

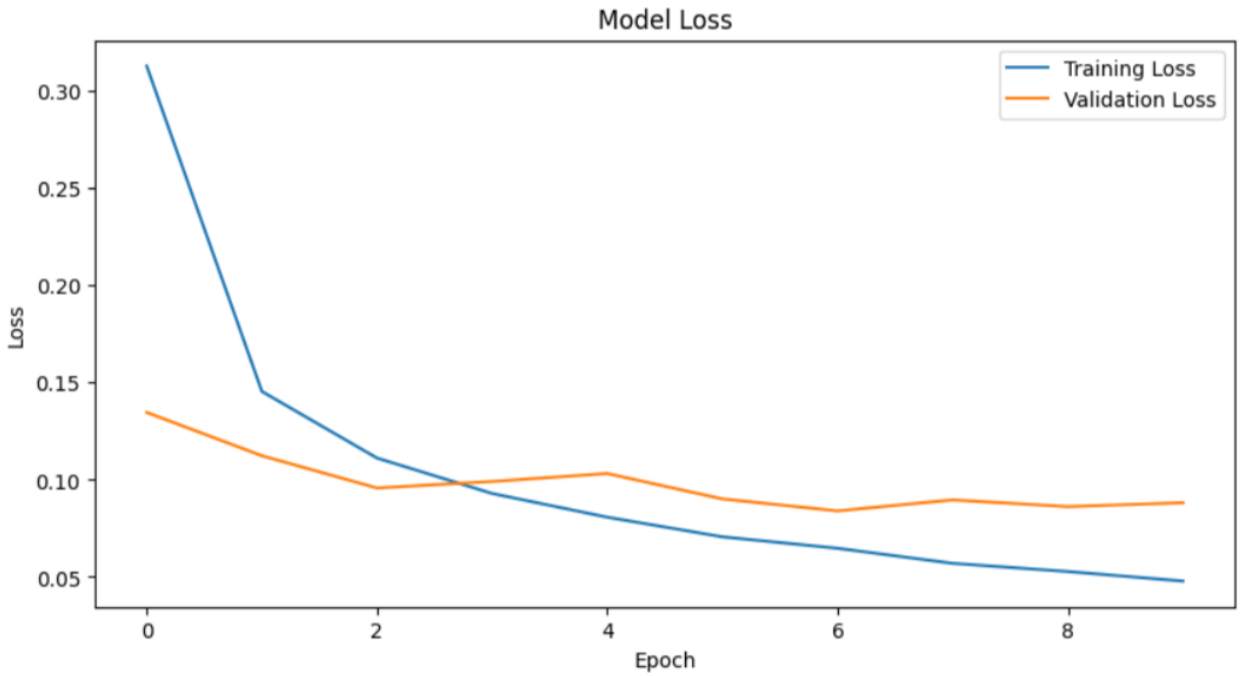
Output:



Step 11: Plot Loss Graph

```
plt.figure(figsize=(10,5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()
```

Output:



Step 12: Evaluate Model

```
test_loss, test_accuracy = model.evaluate(X_test, y_test_cat)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

Output:

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9769
- loss: 0.0789
Test Loss: 0.07892279326915741
Test Accuracy: 0.9768999814987183
```

Step 13: Make Predictions

```
predictions = model.predict(X_test)

predicted_labels = np.argmax(predictions, axis=1)

print(predicted_labels[:10])
```

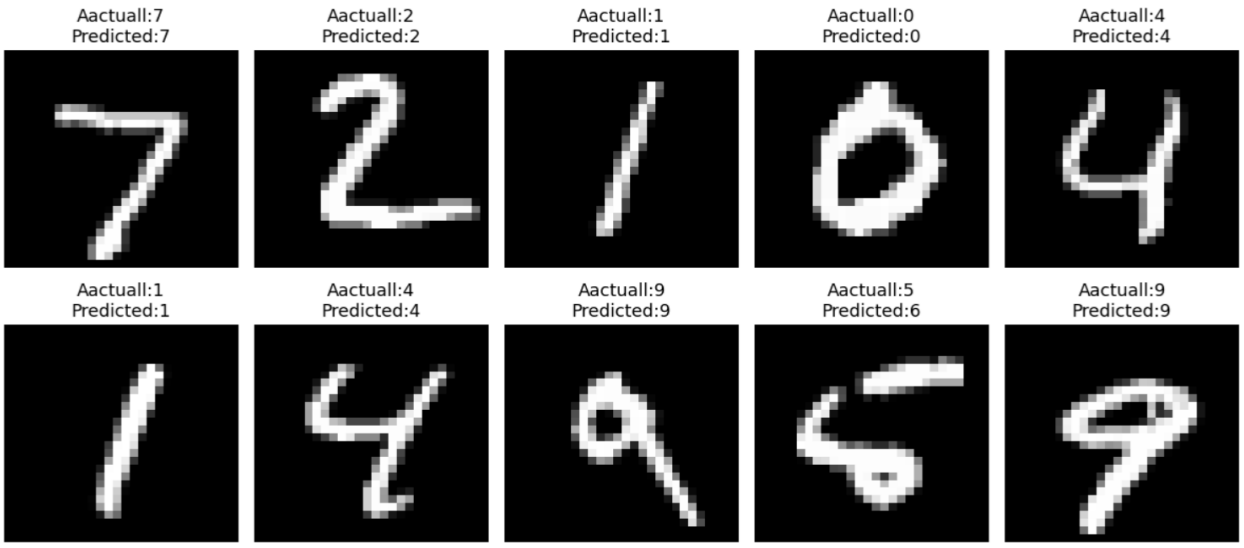
Output:

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
[7 2 1 0 4 1 4 9 6 9]
```

Step 14: Compare Predictions

```
plt.figure(figsize=(12,6))

for i in range(10):
    plt.subplot(2, 5, i+1)  # 2 rows, 5 columns
    plt.imshow(X_test[i], cmap='gray')
    plt.title(f"Aactuall:{y_test[i]}\nPredicted:{predicted_labels[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



Step 15: Confusion Matrix

```
cm = confusion_matrix(y_test, predicted_labels)

plt.figure(figsize=(10,8))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

Step 16: Classification Report

```
print(
    classification_report(
        y_test,
        predicted_labels
    )
)
```

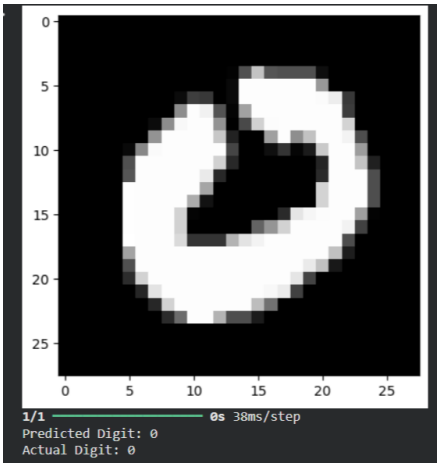
Output:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.98	0.97	0.98	1032
3	0.97	0.98	0.97	1010
4	0.99	0.96	0.98	982
5	0.98	0.97	0.97	892
6	0.98	0.98	0.98	958
7	0.97	0.98	0.98	1028
8	0.97	0.97	0.97	974
9	0.97	0.97	0.97	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

Step 17: Predict a Single Image

```
index = 25

image = X_test[index]

plt.imshow(image, cmap='gray')
plt.show()

prediction = model.predict(
    image.reshape(1,28,28)
)

digit = np.argmax(prediction)

print("Predicted Digit:", digit)
print("Actual Digit:", y_test[index])
```

Output:



Step 18: Save Model

```
model.save("mnist_model.h5")

print("Model Saved Successfully")
```

Output:

```
Model Saved Successfully
```

Step 19: Load Saved Model

```
from tensorflow.keras.models import load_model

loaded_model = load_model("mnist_model.h5")

print("Model Loaded Successfully")
```

Output:

```
Model Loaded Successfully
```

Step 20: Test Loaded Model

```
prediction = loaded_model.predict(
    X_test[0].reshape(1,28,28)
)
```

```
digit = np.argmax(prediction)

print("Predicted:", digit)
print("Actual:", y_test[0])
```

Output:

```
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step
Predicted: 7
Actual: 7
```

Conclusion

This project successfully demonstrates how Deep Learning can be used to automatically recognize handwritten digits using the MNIST dataset. The developed model was trained on thousands of handwritten digit images, allowing it to learn important patterns and features that distinguish one number from another. After training, the model was able to predict unseen handwritten digits with a high level of accuracy, showing the effectiveness of Deep Learning in image recognition tasks.

Throughout this project, important steps such as data preprocessing, normalization, model building, training, testing, and performance evaluation were performed. These steps helped improve the model's ability to learn from the dataset and make accurate predictions. The evaluation results confirmed that the trained model can successfully classify handwritten digits and minimize prediction errors.

The project also highlights the practical applications of Artificial Intelligence in solving real-world problems. Handwritten digit recognition can be used in areas such as bank cheque processing, postal code recognition, automated form processing, document digitization, and data entry automation. By reducing the need for manual work, such systems can save time, improve efficiency, and reduce human errors.

In addition, this project provides a strong foundation for understanding key concepts of Machine Learning, Deep Learning, Computer Vision, and Artificial Intelligence. The knowledge and techniques learned from this project can be extended to more advanced applications, such as handwritten character recognition, object detection, facial recognition, and image classification systems.

Overall, the project demonstrates the power and potential of Deep Learning in modern technology. It shows how computers can be trained to perform tasks that traditionally required human intelligence, making AI-based solutions more practical and valuable in today's digital world. Future improvements may include using more advanced neural network architectures, larger datasets, and real-time image processing systems to further enhance accuracy and performance